JVD Internal Management System

Implementation Roadmap (Master Checklist) · v1.0 FINAL · 2026-07-04

JVD Professionalization — Final Implementation Roadmap (v1.0)

Date: 2026-07-04 · **Status: FINAL** — consolidates and supersedes the sequencing sections of the three companion documents:

Document	Contains
`SYSTEM_AUDIT_AND_REFACTOR_PLAN.md`	Full system scan: what exists, what works (verified by test run), what's missing, DevOps standards, why-not-Kubernetes
`DESIGN_DIRECTION.md`	Figma-derived tokens, 14-component library, modal/drawer/page surface rules, screen migration order
`PRODUCT_IMPROVEMENT_SPEC.md`	Brand colors, notifications, email digests, document repository (DMS), workflow engine, flexible roles, dashboard widgets, sales service catalog, data-integrity rules
`ARCHITECTURE_AND_DATA_REVIEW.md`	Database & architecture verdict: what holds up (decimal money, real FKs, indexes), the four structural flaws (invoice god-table, twin travel tables, dangerous cascades, informal polymorphism), target modular-monolith architecture
`OPERATIONS_TRAINING_AND_ROLLOUT.md`	PH deployment & costs, caching guide, maintenance/IT support model, DPA/BIR compliance, handoff kit, employee training & in-app tutorials
`TEAM_WORKFLOW.md`	**Who does what**: per-item developer assignments (Val · Emman · Greg · Jerald), branch strategy, the one-time branch migration, worktree setup, review rules, weekly rhythm

Ownership: every checklist item below has an assigned Owner + Support in the `TEAM_WORKFLOW.md` §2 matrix. Summary — **Val**: money paths, data integrity, workflow engine, infra, releases (lead, ~60% coding capacity) · **Emman**: platform (CI, queues, notifications, API layer) + QA/test ownership · **Greg**: design system, components, dashboards, HR/Admin/Accounting pages · **Jerald**: data layer, checkout wizard, Sales/Travel/Logistics pages, help center.

This file is the single execution checklist. Work top to bottom; every item links back to its spec. Estimated total: **~14–16 weeks** of focused work for a small team, shippable in slices throughout.

Phase 0 — Stop the bleeding (Week 1)

- [ ] **0.1** Fix `PayrollTest` tax failure — determine if payroll code or test is wrong (**money bug**, do first) *(Audit §3.1)*
- [ ] **0.2** Fix unused-var TS error in `VisaProcessing.tsx:574` so `npm run build` passes

- [ ] **0.3** ⚡ **Wrap the money paths in `DB::transaction` now**: `BillingController::store/update`, Collections payment recording, Liquidations. Days of work, eliminates the worst "data forgotten" failures *(Spec §9 rule 2 — pulled forward from Phase 2 deliberately)*
- [ ] **0.4** Repo cleanup: delete `test*.php`, `php83.zip`, `rr.exe`, diff/output/log/backup junk; move `php83/` + `tools/` out of git; tighten `.gitignore`
- [ ] **0.5** Protect `main`; adopt feature-branch → PR → review; reconcile & retire personal branches (`Emman`, `Val`, `Greg`)
- [ ] **0.6** Consolidate root-level docs into `docs/`; mark stale QA reports superseded

Done when: both builds green, `main` protected, money writes atomic.

Phase 1 — Safety net: CI/CD + environments (Weeks 1–3)

- [ ] **1.1** GitHub Actions: backend (pint + phpunit vs Postgres service) & frontend (eslint + tsc + build), required on PRs
- [ ] **1.2** `docker-compose.yml` dev env (app, postgres, redis, vite) + production `Dockerfile`
- [ ] **1.3** Staging + production environments; auto-deploy staging from `main`, tagged releases to prod; documented rollback
- [ ] **1.4** Sentry (backend + frontend) + uptime monitoring
- [ ] **1.5** Nightly `pg_dump` offsite + one rehearsed restore
- [ ] **1.6** Prod hardening: `APP_DEBUG=false`, seeder env-guards, rotate `password123` accounts
- [ ] **1.7** Start the **sales domain workshops** in parallel — meetings, not code *(Spec §8.1: one per category, capture field worksheets)*

Done when: red PRs can't merge; merge→staging is zero-touch; a test error appears in Sentry; a backup has been restored once.

Phase 2 — Backend correctness & architecture (Weeks 3–8)

Order matters — each item makes the next cheaper:

- [ ] **2.1** Queues live: `app/Jobs/`, all Mailables/Notifications `ShouldQueue`, PDF generation queued, worker under Supervisor/Docker
- [ ] **2.2 Data integrity part 1** *(Spec §9)*: derive invoice payment status from payments (kill hand-set status), `lockForUpdate()` on bus/driver/stock availability checks, idempotency keys on PayMongo webhook
- [ ] **2.2b DB integrity wave** *(Architecture Review §2.3–2.4)*: flip financial/history cascades to `restrictOnDelete`, add the 15 missing FKs, CHECK constraints on amounts, `softDeletes` on master data
- [ ] **2.2c Merge `local_travels` + `international_travels` → one `travels` table** with partial unique indexes on `(bus_id, travel_date)` / `(driver_id, travel_date)` — DB-level double-booking guard *(Architecture Review §2.2)*
- [ ] **2.3 Abilities & roles** *(Spec §6)*: ability registry, roles-as-data, per-user overrides, admin role editor; migrate `role:` scatters as controllers get touched
- [ ] **2.4 Workflow engine** *(Spec §5)*: definitions/steps/instances/actions tables + `WorkflowService`; migrate Cash Budget → PO → WO; approvals inbox endpoint; wire `PublicRequestActionController` email links into it

- [ ] **2.5 Data integrity part 2**: `LedgerService` postings from billing/collections/procurement; finalization snapshots; corrections-as-new-records policy *(Spec §9 rules 3–4)*
- [ ] **2.5b Split `bookings` out of `invoices`** — invoice = immutable financial doc, booking = operational record; one writing module each; the single biggest structural fix *(Architecture Review §2.1)*
- [ ] **2.6 Notifications + email** *(Spec §2–3)*: event-driven, actionable-vs-informational, preferences table, digest engine, central Mailer (ban direct `Mail::to`), Reverb bell channel (delete `ws-server.js`)
- [ ] **2.7 Document repository** *(Spec §4)*: `documents` + polymorphic links + versions + hardcopy tracking + auto-filing listeners + expiry scheduler; backfill old tables
- [ ] **2.8** API standardization: FormRequest per write (kills `validate()` nested-key loss), API Resource per response, pagination everywhere, `throttle:api`, split `routes/api.php` per module, `/api/v1`
- [ ] **2.8b** Status vocabulary: replace drifted enums with CHECK constraints generated from workflow definitions as modules migrate onto the engine *(Architecture Review §3)*
- [ ] **2.9** Thin the god controllers (TripTicket 838 → Billing 616 lines etc.) into `app/Services/` (merge the two service namespaces); Sanctum SPA cookie auth; N+1 audit (`Model::shouldBeStrict()`) + missing indexes
- [ ] **2.10** Backend tests for every money path: billing, collections, liquidations, contracts, commissions, cash budgets, ledger postings

Done when: every money mutation is transactional + ledger-posted + tested; approvals run through one engine; one notification/email pipeline; no controller >~300 lines.

Phase 3 — Frontend: design system + rebuild (Weeks 6–12, overlaps Phase 2)

- [ ] **3.1** Design tokens *(Design §1 + Spec §1)*: brand-semantic palette (blue=interactive, red=danger, amber=warning), typography, radii, motion → Tailwind theme; ESLint ban on raw hex
- [ ] **3.2** Component library *(Design §2)*: AppShell/Sidebar, CommandPalette (⌘K), DataTable, ListRow, StatusPill (+ one `statusColors.ts` mapping), Modal/wizard, Drawer, SharePopover, StatCard/Chart, EmptyState, OnboardingChecklist, Toast — all states, demo route; one icon lib (drop `react-icons`), one toast system (drop `sweetalert2`)
- [ ] **3.3** Surface rules enforced *(Design §2.1)*: toast/confirm/modal/drawer/page decision guide; convert the 22 hand-rolled overlays as pages migrate; no nested modals
- [ ] **3.4** React Query everywhere + route-level code splitting (`React.lazy`) + lazy exceljs/jspdf; initial bundle < 300 KB gz
- [ ] **3.5 Dashboards as widgets** *(Spec §7)*: widget registry + role default layouts + 3 archetype defaults; retire the 9 bespoke dashboard pages and `DashboardController`'s per-role methods
- [ ] **3.6 Service catalog + checkout wizard** *(Spec §8)*: `service_categories` w/ field schemas from the Phase 1 workshops; schema-driven dynamic forms; 5-step checkout; `FixedPackages.tsx` becomes catalog data
- [ ] **3.7** Page-by-page migration: Login/2FA → AppShell → Dashboards → Sales → HR (Employees = DataTable pilot, Applications = ListRow pilot) → Accounting → Travel → the rest; split every >1,000-line page as touched
- [ ] **3.8** Auth UX polish per Figma (first screen every employee sees)

Done when: every page uses the library; no page >~500 lines; dashboards & checkout are data-driven; bundle split.

Phase 4 — Proof (Weeks 12–14)

- [ ] **4.1** Vitest + RTL on the component library & critical hooks
- [ ] **4.2** Playwright E2E: ~10 happy paths (login+2FA, PO chain, checkout→contract→portal-sign, cash budget chain, payroll run, collection payment)
- [ ] **4.3** k6 load test against staging at expected company concurrency; fix findings
- [ ] **4.4** CI coverage gates (backend ratchet; no new untested money controller)

Phase 4.5 — Rollout & training (with/after Phase 4) *(Ops guide §5)*

- [ ] **4.5.1** Patch known dependency CVEs now + add `composer audit` / `npm audit` to CI (monthly cadence)
- [ ] **4.5.2** Runbook (`docs/RUNBOOK.md`): deploy, rollback, restore, rotate — tested by someone who didn't write it; access + env inventories in a password manager
- [ ] **4.5.3** Help Center in-app: FAQ articles (seeded from pilot feedback), "Report a problem" → IT Request workflow
- [ ] **4.5.4** In-app onboarding: role-specific first-login checklists + spotlight tooltips + "new feature" popovers (components from Design §2 #11–12)
- [ ] **4.5.5** Pilot → champions → module go-lives with 2-week feedback windows; per-role hands-on hour in staging; 3–5-min task videos
- [ ] **4.5.6** Compliance: DPA privacy notice on portal/KYC, DPO named, retention configured in DMS; BIR CAS conversation with the accountant

Phase 5 — Scale & polish (ongoing)

- [ ] S3-compatible file storage · Redis caching for dashboards/reports · log aggregation · Octane decision · second app server only if usage demands (still no Kubernetes)
- [ ] `php artisan schema:dump` to squash the 140-migration history into one baseline; optional `app/Modules/` reorganization *(Architecture Review §5)*

Learning track — "the right processes" (for the team, alongside the work)

Each phase *is* the curriculum; learn by doing it on your own system:

While doing	You learn	Reference
Phase 0–1	Git flow/PRs, CI/CD, Docker, environments, backups	Audit §5
2.1–2.2	Queues, transactions, race conditions, idempotency	Spec §9 (the six rules — read this one twice)
2.3–2.5	RBAC design, state machines, double-entry ledger, immutability/snapshots	Spec §5, §6, §9
3.1–3.3	Design tokens, component-driven UI, UX surface patterns	Design doc
Phase 4	Testing pyramid, E2E, load testing	Audit §5.2

The habit that replaces vibe-coding: **model first, screen last** — workshop the domain (Spec §8.1), write the data shape and its rules, then let screens render the model. Fields, statuses, and colors all become *derived from definitions* instead of guessed per page.

Priority cheat-sheet (if you can only do five things this month)

1. **0.1 + 0.3** — payroll bug + transactions on money paths (correctness)
2. **0.5 + 1.1** — protected main + CI (stops new regressions)
3. **1.3–1.5** — staging, Sentry, backups (operational survival)
4. **2.2** — derived payment status + locks (kills "switched/overwritten" data)
5. **1.7** — sales workshops (unblocks the entire sales rebuild, costs no code)